

Otonom Araç Simülatörü

BLM4502 – YAPAY SİNİR AĞLARINA GİRİŞ

Proje Ekibi

1221602020 – Emirhan Civelek
1221602031 – Mert Can Aydın
1221602069 – Emre Çalışkan
1221602066 – Furkan Çelik
1211602018 – Umut Diler

İçindekiler

- Projenin Konusu ve Amacı
- Projeyi Neden Seçtik?
- Takviyeli Öğrenme Nedir?
- Temel Takviyeli Öğrenme Kavramları
- Takviyeli Öğrenmeyi Neden Seçtik?
- Projenin Katmanları ve Çalışma Şekli
 - Model Eğitiminde Kullandığımız Algoritmalar
 - DQN
 - PPO
 - A2C
 - SAC
 - Frontend
 - Kullanıcı Arayüz Ekran Görüntüleri
 - Backend
 - Websocket Nedir ve Önemi
- Eğitilen Modellerin Anlatımı ve Farkları
- Genel Model Karşılaştırması

Projenin Konusu ve Amacı

Bu proje, statik ve hareketli engeller barındıran dinamik ortamlarda otonom navigasyon probleminin Derin Takviyeli Öğrenme (RL) yöntemleriyle çözülmesini konu alır.

Temel amaç; klasik algoritmalar yerine deneme-yanılma yoluyla öğrenen bir yapı kurmak, DQN, A2C ve PPO gibi farklı RL mimarilerinin performanslarını somut metriklerle analiz etmek ve bu süreci modern bir 2D/3D web arayüzünde gerçek zamanlı olarak görselleştirmektir.



Bu Proje Konusunu Neden Seçtik?

Projeyi seçmekteki amacımız , otonom şekilde ilerleyen araçların hata yapmadan nasıl ilerleyeceğini fiziksel testlerle yapan firmaların aslında bu denli bir proje ile 2D boyutlarla bir web sitesi üzerinden otonom araca ya da robot için eğittikleri modelleri maliyet açısından 0'a düşürerek nasıl yapılabileceğini görmek istememizden.

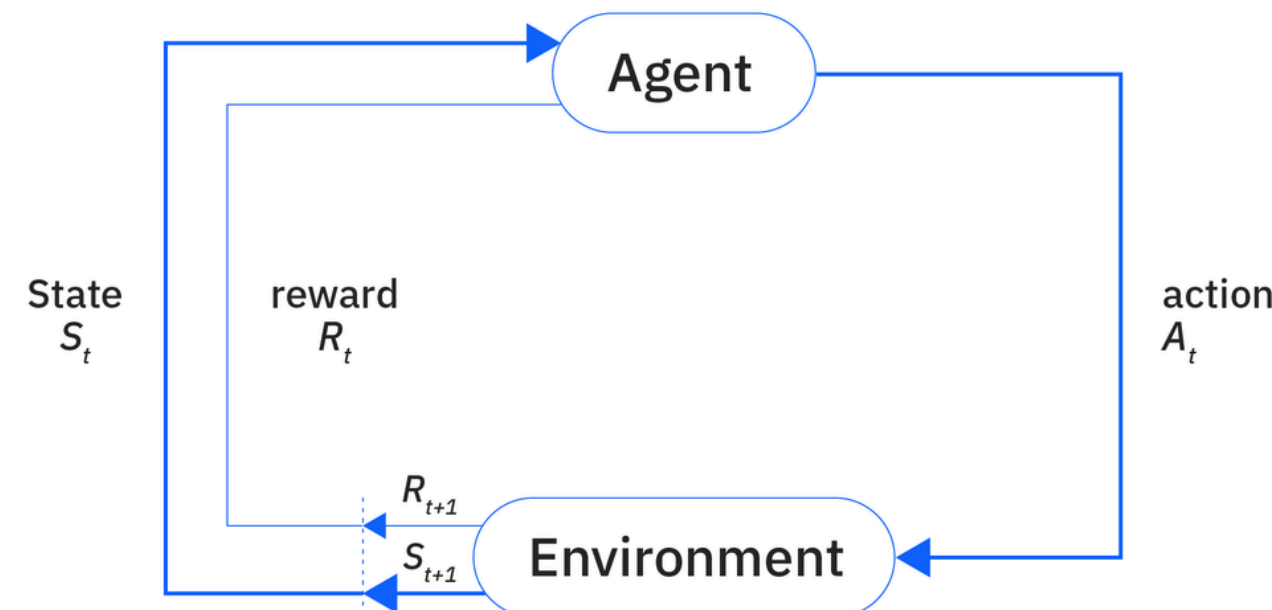
Bu tarz bir web arayüzünün aşağıdaki gibi avantajları olacağını da ön görerek bu projeyi yapaya karar verdik :

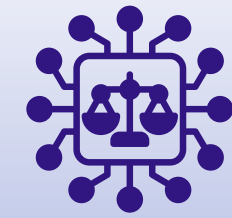
- Modelin karşılaşılabileceği her türlü zorlu engel durumunu fiziksel olarak zor olabilir , bu şekilde tek tıkta modelin bulunduğu bağlamı değiştirebiliyoruz.
- Birden fazla modeli eş zamanlı deneyebilmek ve birbirine göre avantajlarını kıyaslayabilmek
- 3D şekilde simüle edebilmek

Takviyeli Öğrenme Nedir?

Takviyeli öğrenme (RL), ajanın çevreyle doğrudan etkileşime girerek edindiği deneyimlerden en uygun davranışı kendi kendine öğrenmesini sağlar. Bu süreç, ajanın hedefe yönelik doğru adımlarında puan kazandığı, hatalı kararlarında ise puan kaybettiği bir ödül-ceza mekanizmasına dayanır.)

Ajanın nihai amacı, anlık kazançlardan ziyade uzun vadede elde edeceği kümülatif (toplam) ödülü maksimize edecek en ideal stratejiyi, yani optimum politikayı keşfetmektir.





Temel RL Kavramları

Durum / Gözlem (s): Ajanın (agent) belirli bir t anında içinde bulunduğu ortamı, kendi koşullarını ve çevresini tanımlayan anlık veri veya bilgi setidir.

Aksiyon (a): Ajanın mevcut durumunda, çevresiyle etkileşime geçmek veya durumunu değiştirmek için gerçekleştirebileceği olası tüm hamlelerin (kararların) kümesidir.

Ödül (r): Ajanın seçtiği bir aksiyon sonucunda çevreden anında aldığı sayısal (skaler) geri bildirimdir. Bu sinyal, yapılan hamlenin problemin amacına ne kadar uygun olduğunu gösterir.

Politika (pi): Ajanın "hangi durumda hangi aksiyonu alması gerektiğini" belirleyen matematiksel strateji, kural veya olasılık dağılımıdır. Ajanın nihai amacı, toplam ödülü maksimize edecek en iyi (optimum) politikayı bulmaktır.

İndirim Faktörü (gamma): Gelecekte elde edilecek ödüllerin, anlık ödüllere kıyasla ne kadar önemseneceğini belirleyen ve 0 ile 1 arasında değer alan (genellikle 0.90 – 0.99) hiperparametredir. Ajanın kısa vadeli kazançlar ile uzun vadeli hedefler arasındaki dengesini ayarlar.

Takviyeli Öğrenmeyi Neden Seçtik?

Takviyeli Öğrenmeyi seçmekteki amacımız projenin anabasığı olan otonom sürüşleri simüle etmemizden kaynaklı oluyor aslında.

Otobanda giden bir aracı düşünelim , bu araç belli bir görüş mesafesi ve çeşitli sensörlerle , ki bu sensörler aracın nasıl bir eylem içinde olacağına karar vermesindeki girdileri sağlar , Hedefini ya da amacını otomatik olarak gerçekleştirmeye çalışır.

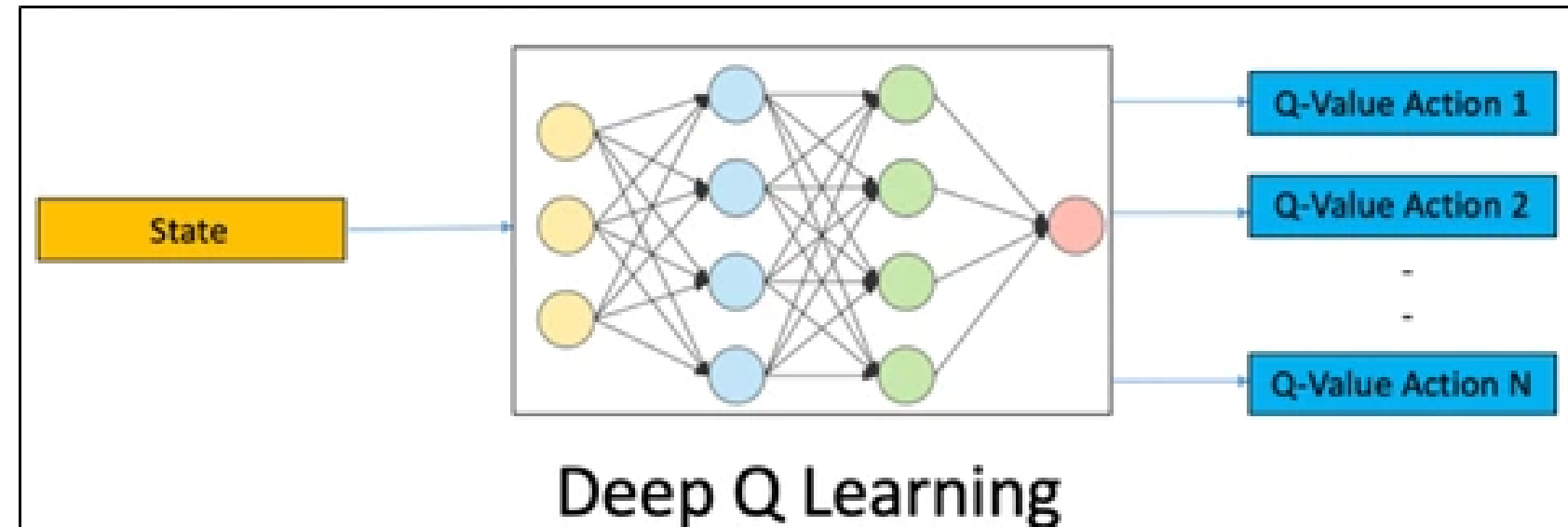
Bizim projemiz için ise bu tarz bir otonomun bir şekilde engelleri tanıyarak belli bir hedef doğrultusunda ilerlemek için en doğru hamleleri yapabilmesini Takviyeli Öğrenme metoduyla gerçekleştirebileceğı kanaatindeyiz.

Projenin Katmanları ve Çalışma Şekli

Proje eğitilen modelleri simüle edebileceğimiz bir arayüz(websitesi) bu arayüze modellerden gelen cevapları anlık olarak arayüze response olarak dönen bir backend ve tabii ki asıl önemli parça olan modellerden oluşur.

DQN (Deep Q-Network)

Geleneksel takviyeli öğrenme algoritmaları (Q-Learning), aracın karşılaşılabileceği tüm durumları bir tabloya kaydeder. Ancak otonom sürüşte; kamera görüntüleri, şerit bilgileri ve engeller gibi milyonlarca farklı kombinasyon (durum uzayı) bulunur. Bu kadar devasa veriyi bir tabloda tutmak imkansızdır. DQN, bu problemi çözmek için tablo kullanmak yerine bir Yapay Sinir Ağı yerleştirir. Araç, durumları hafızada tutmaz; derin sinir ağını kullanarak hangi durumda hangi hareketi yapması gerektiğini tahmin etmeyi öğrenir.



DQN Nasıl Çalışır

Algoritma, web simülasyonu ortamında sürekli dönen 4 aşamalı bir döngüden oluşur:

1. **Gözlem (State):** Araç, web tarayıcısı üzerinde çalışan simülasyondan anlık verileri toplar (Örn: Sanal sensörlerin engellere uzaklığı, şerit çizgilerine olan mesafe ve hız).
2. **Eylem Seçimi (Action):** Sensör verileri yapay sinir ağına gönderilir. Ağ; Hızlan, Fren Yap, Sağa Dön, Sola Döneylemleri için başarı skorları (Q-değerleri) üretir. Araç en yüksek skora sahip hamleyi seçer.
3. **Ödül Fonksiyonu (Reward):** Seçilen hamle simülasyonda uygulanır. Araç şeritte düzgün ilerledikçe pozitif ödül (+), şeritten çıktıkça veya engellere çarptıkça negatif ceza (-) alır.
4. **Deneyim Hafızası (Experience Replay):** Araç, web üzerinde yaptığı her sürüşün deneyimini bir hafıza havuzunda saklar. Bu havuzdan rastgele seçilen verilerle sinir ağı sürekli optimize edilir ve araç geçmiş hatalarından ders çıkarır.

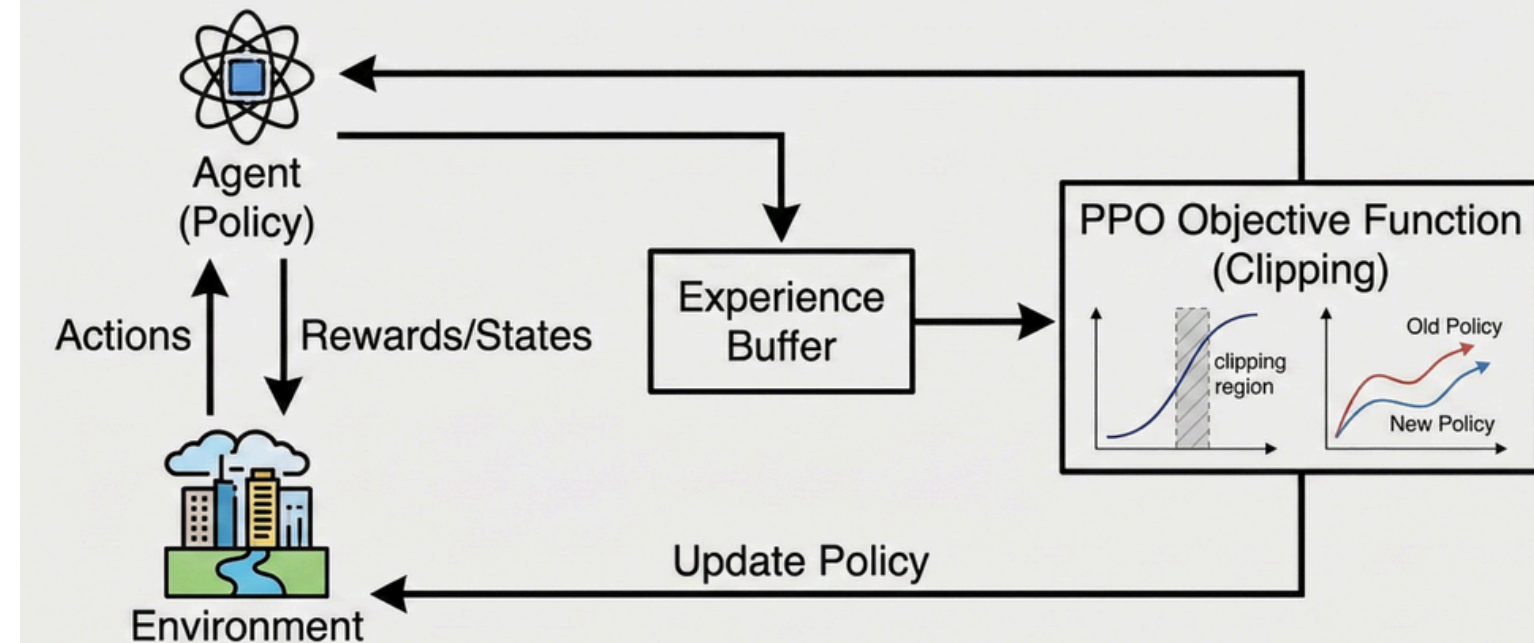
PPO (Proximal Policy Optimization)

PPO, OpenAI tarafından 2017 yılında geliştirilen, yapay zekanın deneme-yanılma yoluyla karar almayı öğrendiği gelişmiş bir Takviyeli Öğrenme (Reinforcement Learning) algoritmasıdır.

Günümüzde yürüyen robotlardan ChatGPT'nin insanlarla doğru konuşmayı öğrenmesine (RLHF) kadar dünyadaki en popüler yapay zeka yönlendirme mekanizmalarından biridir.

Temel amacı; yapay zekanın veri toplama ve kendini güncelleme sürecini en güvenli, en kararlı seviyede tutmaktır.

Proximal Policy Optimization (PPO) Algorithm

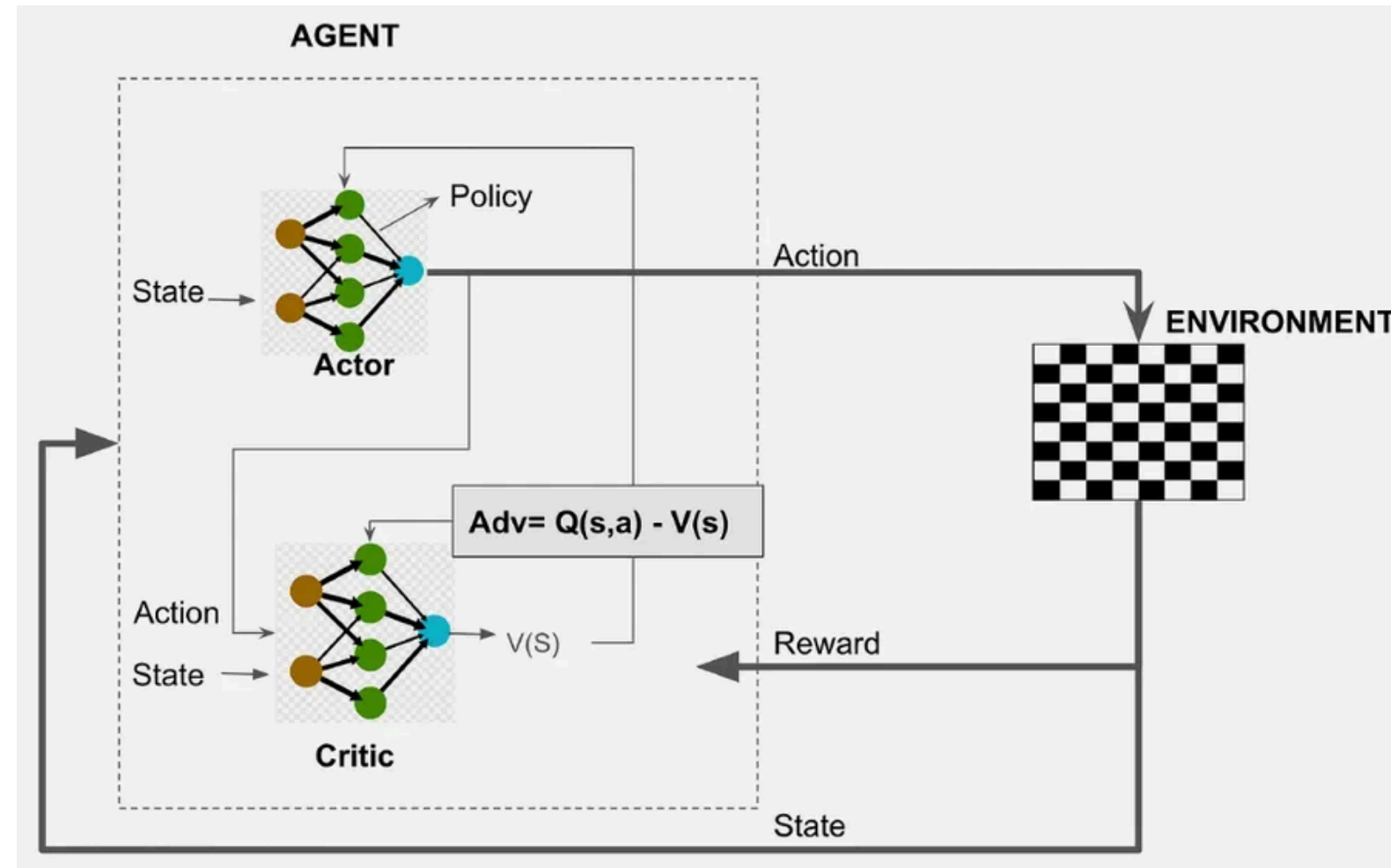


Neden PPO'yu Seçtik ve Önemli Mekanizmalar

- PPO algoritmasının kararlı ve pürüzsüz bir şekilde öğrenmesini sağlayan şey, arkasındaki matematiksel emniyet kemeri olan Kırpma (Clipping) mekanizmasıdır.
- **Ani Değişimlerin Engellenmesi:** Yapay zeka simülasyonda denemeler yaparken bazen ani ve çok radikal kararlar almak isteyebilir. PPO, yeni sürüş stratejisinin eski başarılı stratejiden en fazla %20 oranında sapmasına izin verir; daha büyük adımları otomatik olarak kırpar (sınırlandırır).
- **Kazanılmış Yeteneklerin Korunması:** Bu sınırlama sayesinde yapay zeka, o ana kadar öğrendiği iyi sürüş alışkanlıklarını (şeritte kalma, virajı dönme vb.) tek bir hatalı adımla aniden unutmaz.
- **Sonuç:** Eğitim süreci büyük dalgalanmalar veya ani çöküşler yaşamadan, her adımda mevcut sürüş yeteneğinin üzerine koyarak kararlı ve güvenli bir şekilde başarıya ulaşır.

Senkron A2C (Aktör-Eleştirmen)

A2C (Advantage Actor-Critic), tek bir sinir ağı üzerinde iki farklı görevi eş zamanlı yürüten bir aktör-eleştirmen mimarisine dayanmaktadır. Ağın aktör başlığı, ajanın hangi aksiyonu seçeceğini belirleyen politikayı üretirken; eleştirmen başlığı, mevcut durumun ne kadar değerli olduğunu tahmin etmektedir.



Senkron A2C (Aktör-Eleştirmen) Nasıl Çalışır ?

1. Aktör (Actor) – Karar Mekanizması

Direksiyondaki sürücüdür. Simülasyondaki duruma (State) bakar ve bir eylem (Action) seçer. (Örn: "Önümde engel var, sola dönmeliyim.")

2. Eleştirmen (Critic) – Değerlendirme Mekanizması

Yan koltuktaki sürücü kursu öğretmenidir. Aktörün yaptığı hamleyi inceler ve çevreye bakarak bu hamlenin gelecekte ne kadar ödül (Reward) getireceğini tahmin eder.

3. Avantaj (Advantage) – Gelişme Ölçütü

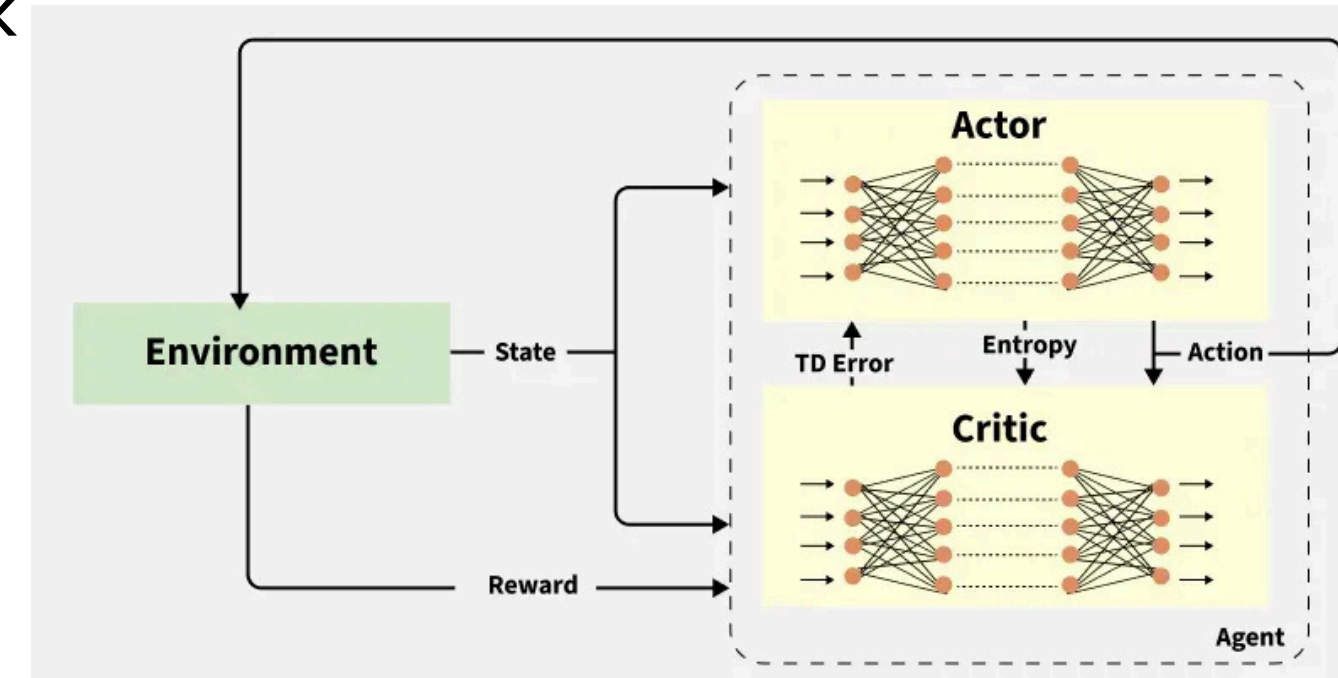
A2C'yi özel kılan matematiksel değerdir. Eleştirmen, aktörün yaptığı hamleyi değerlendirirken şu soruyu sorar: "Bu hamle, bizim beklentimizden daha mı iyi sonuç verdi, yoksa daha mı kötü?" * Eğer hamle beklenenden iyiye Avantaj (A) pozitifdir ve Aktör o hareketi daha sık yapmak üzere kendini günceller.

SAC (Soft Actor Critic)

SAC (Soft Actor-Critic), 2018 yılında UC Berkeley araştırmacıları tarafından geliştirilen, yapay zekanın deneme-yanılma yoluyla karar almayı öğrendiği oldukça esnek ve verimli bir Takviyeli Öğrenme algoritmasıdır.

Günümüzde karmaşık otonom sürüş sistemlerinden gerçek dünyadaki robot kollarının hassas nesne tutma hareketlerini öğrenmesine kadar, özellikle çok fazla deneme yapmanın zor veya maliyetli olduğu robotik sistemlerde dünyadaki en popüler yönlendirme mekanizmalarından biridir.

Temel amacı; yapay zekanın sadece yüksek ödüle odaklanıp aynı hareketleri ezberlemesini engellemek ve "entropi" adı verilen bir yaratıcılık/rastgelelik payı ekleyerek sürekli yeni yollar keşfetmesini ve daha az veriyle en başarılı davranışları bulmasını sağlamaktır.



SAC (Soft Actor Critic) *Nasıl Çalışır?*

1. Aktör (Actor) – Yaratıcı Sürücü Direksiyondaki sürücüdür. Çevresel duruma (State) bakar ve bir eylem (Action) seçer. Ancak geleneksel sürücülerden farklı olarak katı kurallara bağlı kalmaz; her zaman bildiği en garanti yolu seçmek yerine duruma göre farklı ve esnek manevralar yapmaya (stokastik davranmaya) eğilimlidir.

2. İkiz Eleştirmen (Twin Critic) – Denetleme ve Puanlama Yan koltuktaki ve arka koltuktaki iki farklı direksiyon eğitmenidir. Sürücünün yaptığı hamleyi ve gidişatını puanlarlar (Q-Değeri). İki kişi olmalarının sebebi, tek bir öğretmenin aşırı iyimser (overestimation) puanlar verip sürücüyü kandırmasını engellemektir. Her zaman bu iki öğretmenin verdiği en düşük puan dikkate alınır, böylece daha güvenli ve gerçekçi öğrenme sağlanır.

3. Entropi (Entropy) – Esneklik ve Keşif Ölçütü SAC'yi diğerlerinden ayıran en özel kavramdır. Sürücü sadece "En yüksek puanı nasıl alırım?" diye düşünmez, aynı zamanda "Hedefe ulaşırken başka hangi alternatif yolları keşfedebilirim?" diye sorar. Sistem, sürücüyü sadece başarılı olduğu için değil, aynı zamanda tahmin edilemez (yüksek entropili) ve farklı çözüm yolları denediği için de ekstra ödüllendirir. Bu sayede yapay zeka tek bir ezber davranışa saplanıp kalmaktan kurtulur.

Frontend Katmanı

Uygulamanın görsel arayüzü React ile geliştirilmiş, üç boyutlu simülasyon sahnesi için ise Three.js kütüphanesi kullanılmıştır. Kullanıcı arayüzü şu kontrolleri sunar:

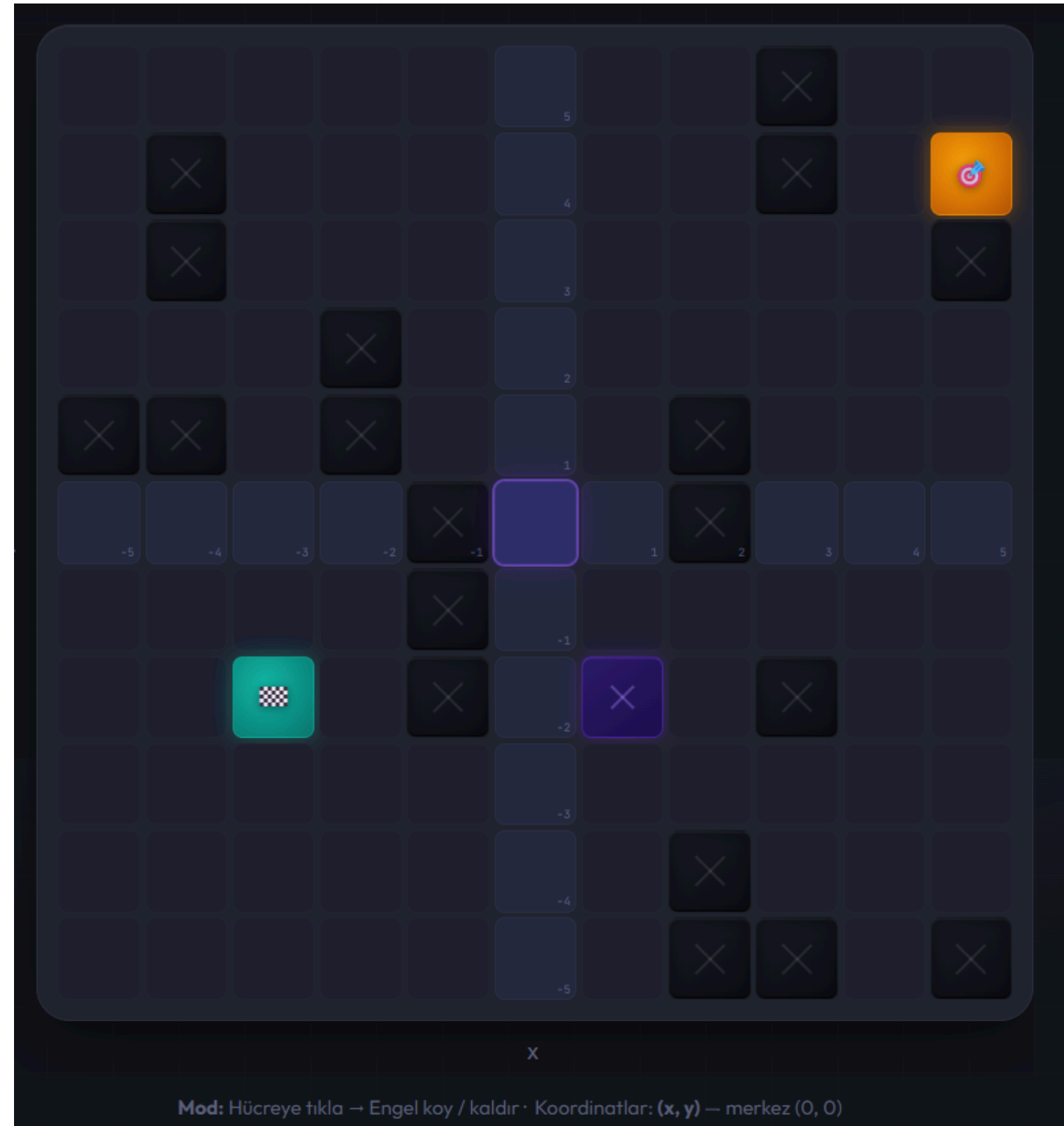
- **Izgara boyutu:** 4 farklı seçenek arasından seçim yapılabilir — 11×11 , 15×15 , 21×21 ve 31×31 . Boyut büyüdükçe harita karmaşıklığı ve dinamik engel sayısı da artar.
- **Zorluk seviyesi:** Rastgele, Kolay (yalnızca statik engel), Orta (1 dinamik), Zor (3 dinamik), Çok Zor (5 dinamik) ve Cehennem (8 dinamik) olmak üzere 6 zorluk kademesi mevcuttur.
- **Harita teması:** Şehir, Depo, Mars ve Hastane olmak üzere 4 farklı görsel tema seçilebilir. Tema yalnızca görseli etkiler, simülasyon mantığını değiştirmez.
- **Model seçimi:** Backend'deki mevcut modeller (DQN, A2C, PPO varyantları) açılır menüden seçilebilir. Seçim yapıldığında backend sunucu yeniden başlatılmadan canlı olarak model değişir.
- Her simülasyon adımında WebSocket bağlantısı üzerinden ajanın yeni konumu, aldığı ödül, Q-değerleri ve adım sayısı gerçek zamanlı olarak ekrana yansıtılır; simülasyon hedefe ulaşma veya çarpışmayla sonuçlandığında kullanıcıya anlık bildirim gösterilir.

Kullanıcı Arayüzü

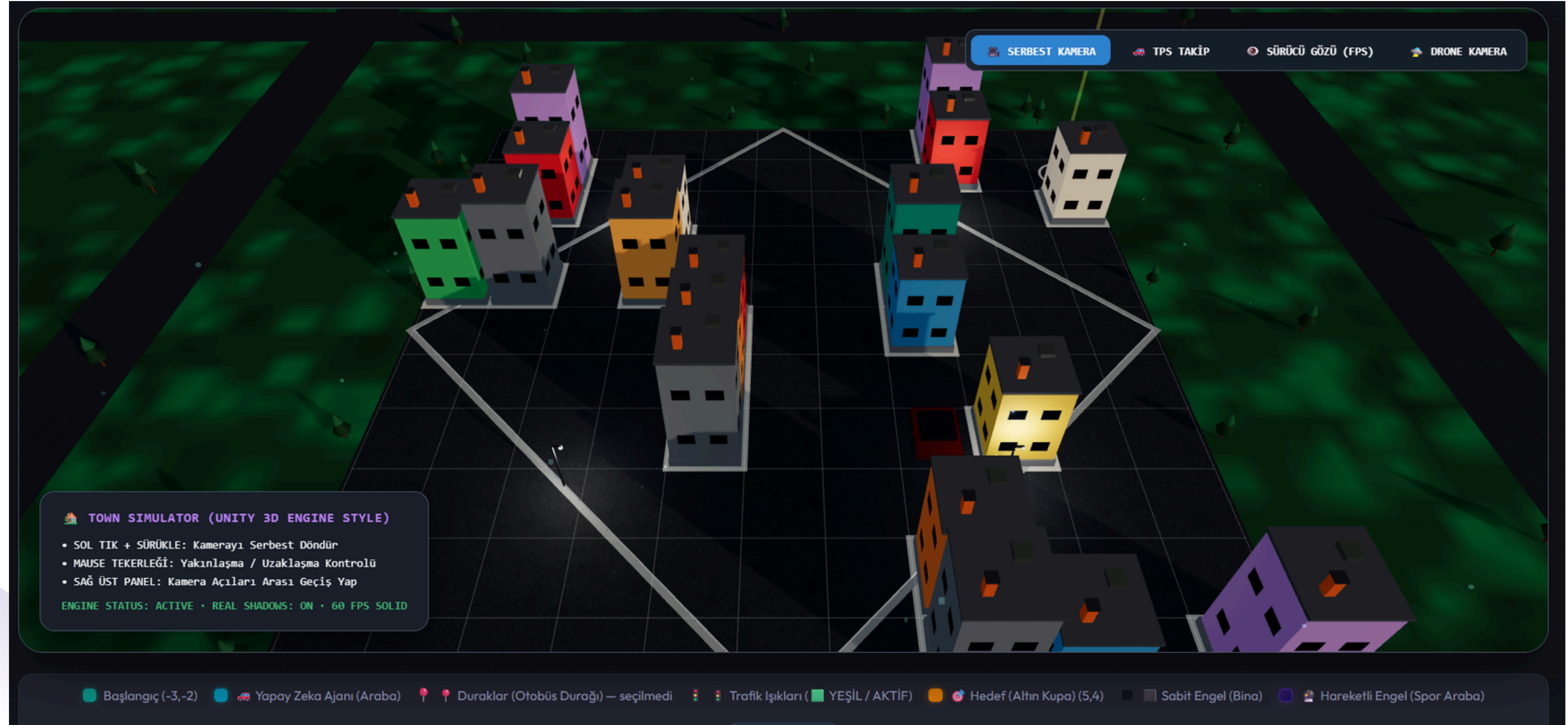


Kullanıcının harita tasarlamak, rastgele harita seçmek ve modeli belirlemek için kullandığı seçeneklerin bulunduğu card.

2 Boyutlu Gösterim-Harita Izgara Yapısı



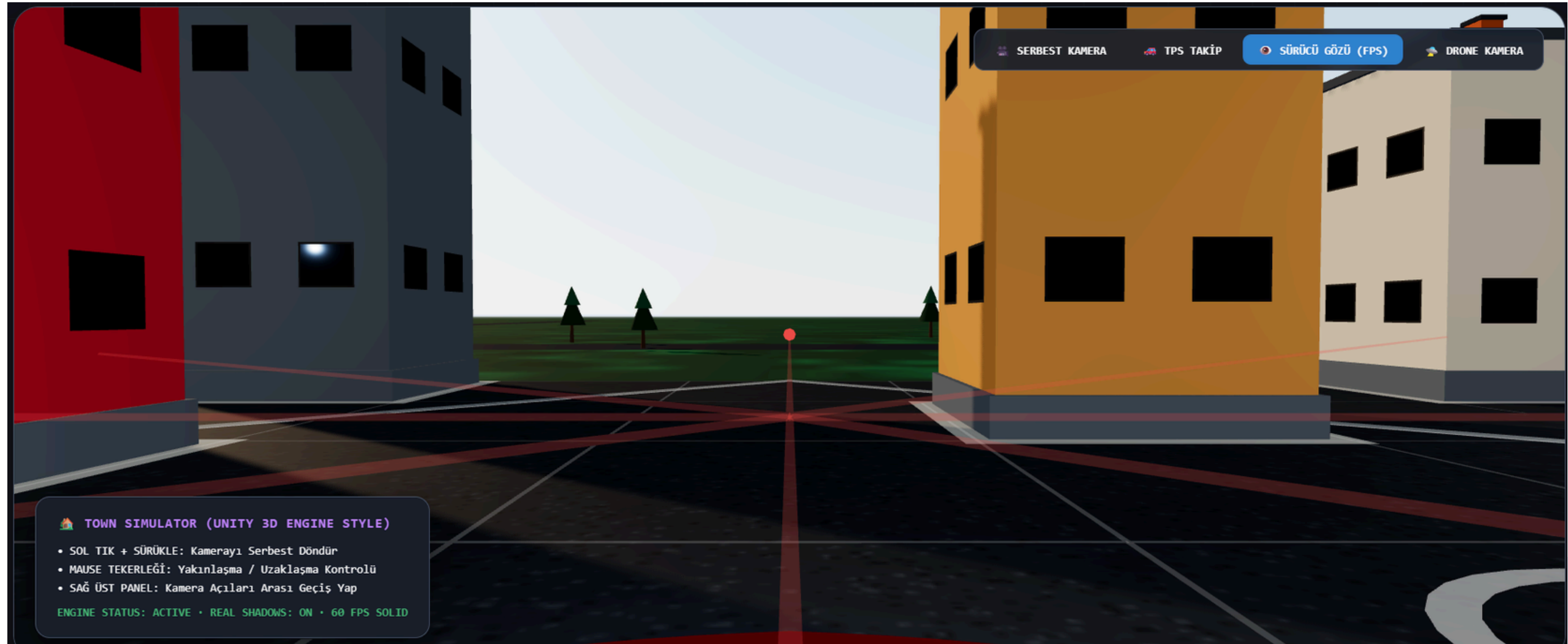
3 Boyutlu Gösterim - Serbest Kamera



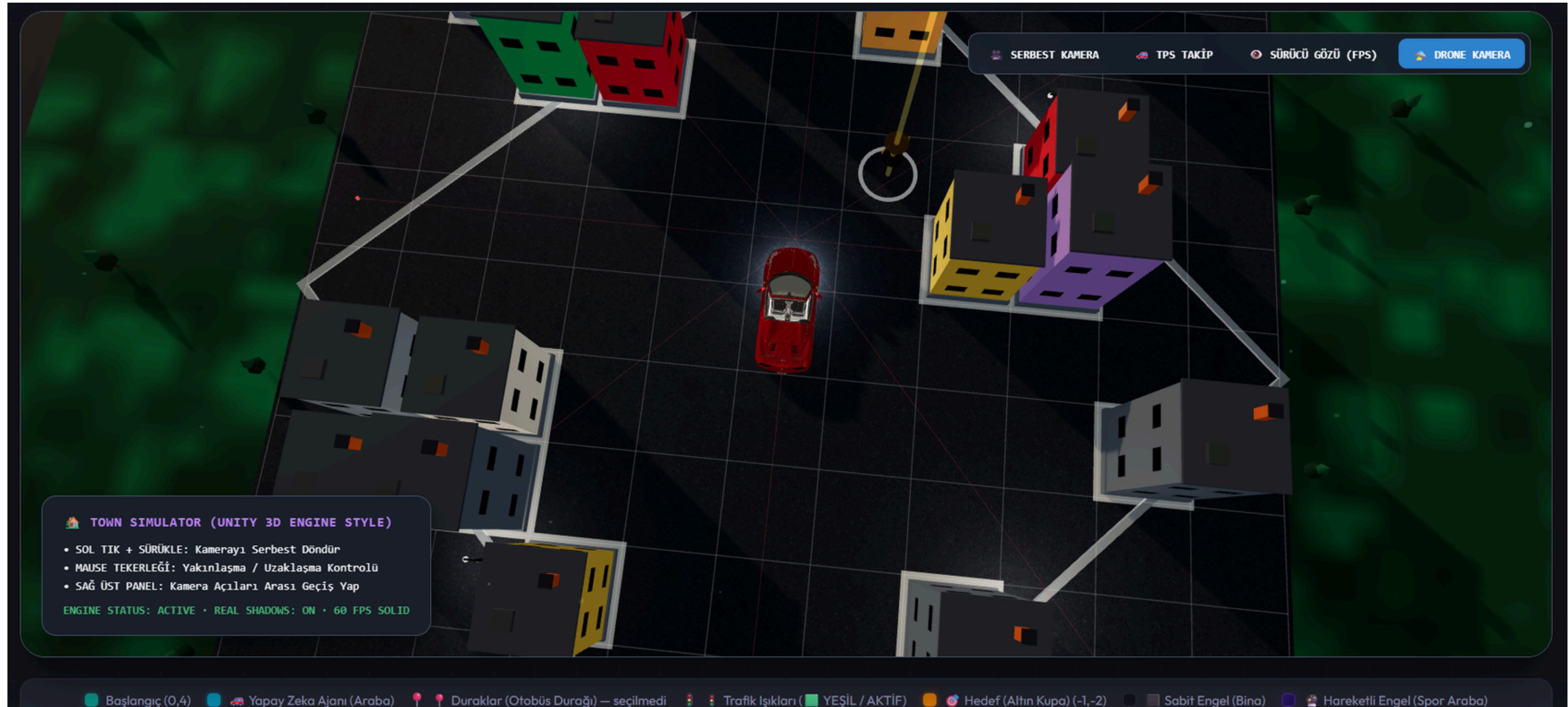
3 Boyutlu Gösterim - TPS Kamera



3 Boyutlu Gösterim - FPS Kamera



3 Boyutlu Gösterim - Drone Kamera



Yarışma Gösterimi

 **AJAN YARIŞI**

5 farklı otonom sürüş modelini aynı pistte yarıştıran! Ajanlar birbirini dinamik engel olarak görüp kaçınacaktır.

 Yarış Modunu Kapat

 **Racer 1**

A3C - A2C_V3

 **Racer 2**

DQN - BEST_MODEL_V4

 **Racer 3**

PPO - PPO_HARDCORE_V2

 **Racer 4**

SAC - SAC_LOCAL

 **Racer 5**

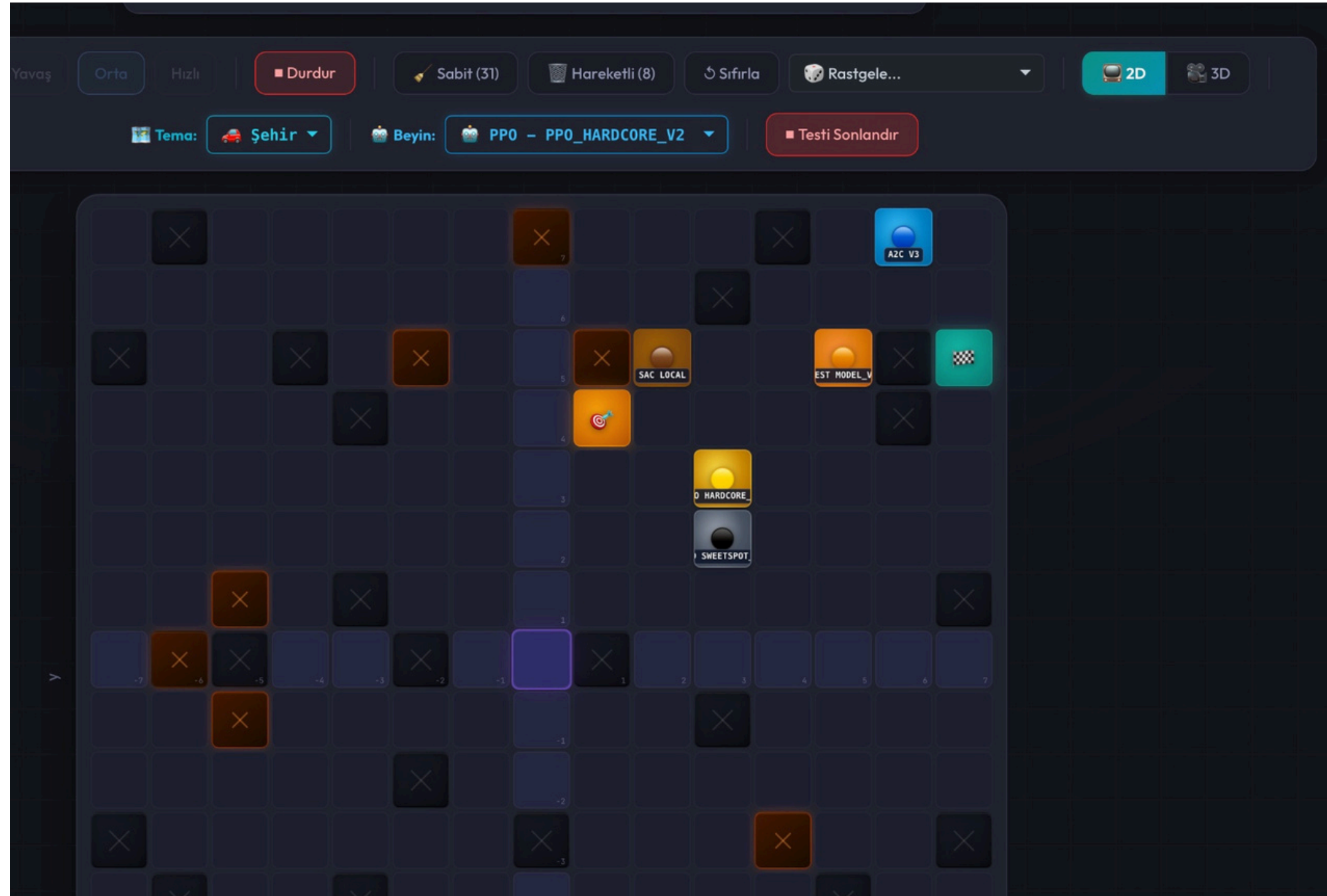
PPO - PPO_SWEETSPOT_V4

 **Yarış Pisti:**

Aktif: **Pist 5**

 **Pist Oluştur**

Yarışma Gösterimi



Backend Katmanı

Bu projede backend, FastAPI framework'ü ve PyTorch kütüphanesi kullanılarak Python ile geliştirilmiş bir yapay zeka servis katmanıdır. Görevleri üç başlıkta özetlenebilir: eğitilmiş sinir ağı modellerini belleğe yüklemek ve yönetmek, simülasyon ortamının (GridEnvironment) durumunu tutmak ve her adımda aksiyon hesaplamak, sonuçları arayüze iletmek.

Neden Websocket?

Geleneksel web mimarisinde (HTTP) iletişim "sor-cevap" mantığıyla çalışır. Yani arayüzün sunucuya sürekli, saniyede defalarca "Ajan hareket etti mi? Yeni koordinat ne?" diye sorması gerekir. Saniyede onlarca karar alan otonom bir yapay zekâ sisteminde bu sürekli sorma döngüsü (polling), sistemi yorar, tıkanmalara yol açar ve asla gerçek zamanlı (real-time) bir akış sağlayamaz.

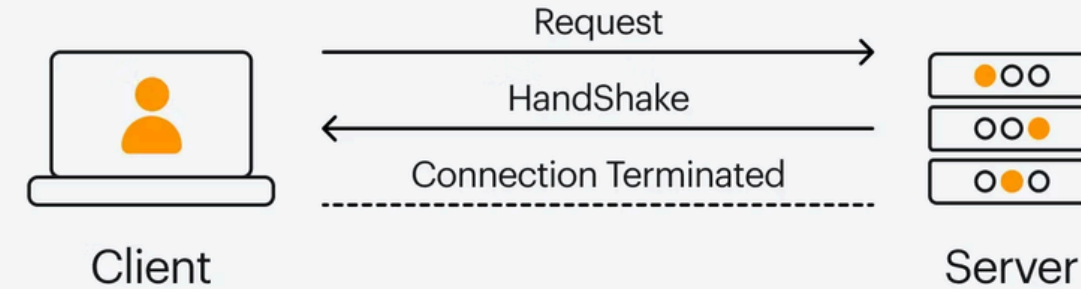
WebSocket, sunucu (yapay zekâ) ile arayüz arasında sürekli açık kalan, iki yönlü bir otoban kurar. Bu otoban sayesinde arayüzün sürekli soru sormasına gerek kalmaz. Yapay zekâ ajanı arka planda bir adım attığı veya bir engele çarptığı an, sunucu bu veriyi beklemeden doğrudan arayüze fırlatır (push). Kısacası simülasyonun kesintisiz, anında ve milisaniyelik gecikmelerle canlı olarak akabilmesinin tek yolu bu teknolojidir.

WebSocket Connection



VS

Other Communication Protocols



Kısacası WebSocket ile Çözüm: Bağlantı çift yönlü olduğu için hem tarayıcı hem de yapay zeka modeli istedikleri anda, birbirlerini beklemeden ve hattı meşgul etmeden veri gönderebilirler.

Model Anlatımları Ve Farkları

DQN ile Eğittiğimiz Model

Değer tabanlı öğrenme paradigmasının bu projedeki temsilcisi olan DQN modeli, Double Dueling mimarisiyle geliştirilmiş ve ϵ -greedy keşif stratejisi uygulanmıştır. Eğitim sürecinde epsilon değeri 1.0'dan 0.01'e kademeli olarak düşürülmüş; model başlangıçta rastgele keşif yaparken zamanla öğrendiklerini sömürmeye başlamıştır.

Öğrenme katsayısı 0.001, indirim faktörü $\gamma=0.99$ olarak belirlenmiş; deneyimler 30.000 kapasiteli bir bellek tamponunda (replay buffer) saklanarak rastgele örnekleme yoluyla eğitim gerçekleştirilmiştir. Ödül şemasında hedefe ulaşma +100, engele çarpma -50, harita dışına çıkma -10 olarak tanımlanmış; hedefe yaklaşma +2.0, uzaklaşma -0.5 ve her adıma -0.1 temel ceza uygulanmıştır. Küçük haritalarda işlevsel sonuçlar üretebilmektedir; ancak harita boyutu arttıkça performansının belirgin biçimde gerilediği gözlemlenmiştir.

ϵ -greedy (epsilon-greedy)

Ajanın keşif ile sömürü arasındaki dengeyi yönetme stratejisidir. Epsilon olasılığıyla rastgele bir aksiyon seçilir (keşif), kalan olasılıkla ise o ana kadar öğrenilmiş en iyi aksiyon uygulanır (sömürü). Eğitim başında epsilon yüksektir — ajan çok keşfeder; zamanla düşer — ajan öğrendiklerini kullanmaya başlar.

PPO ile Eğittimiz Model

PPO_Hardcore_V2 Modeli :

Proximal Policy Optimization algoritmasıyla Stable-Baselines3 kütüphanesi üzerinden eğitilmiş olan bu model, 5 birimlik görüş yarıçapıyla çevresini geniş bir perspektiften değerlendirmektedir.

Öğrenme katsayısı 3×10^{-4} , indirim faktörü $\gamma=0.99$, GAE lambda $\lambda=0.95$, entropi katsayısı 0.015 ve kırpma parametresi clip=0.2 olarak belirlenmiştir.

PPO modelleri farklı bir ödül şemasıyla eğitilmiştir: sabit adım cezası -0.05, yeni hücre keşfine +0.3, tekrar ziyarete kademeli ceza, dinamik engele tehlikeli yakınlıkta -0.2 ve çarpışmada -50 uygulanmıştır. Harita boyutu büyüdükçe başarı oranındaki düşüş son derece sınırlı kalmış, tüm senaryolarda tutarlı ve güvenilir bir performans sergilenmiştir.

λ (lambda)

GAE'de ne işe yarar? GAE (Generalized Advantage Estimation), ajanın bir aksiyonun ne kadar iyi olduğunu ölçmek için kullanılan avantaj hesaplama yöntemidir. Lambda bu hesaplamada kısa vadeli ve uzun vadeli ödüller arasındaki dengeyi ayarlar. $\lambda=0$ 'da ajan yalnızca bir sonraki adıma bakar (düşük varyans, yüksek önyargı); $\lambda=1$ 'de tüm gelecekteki ödülleri hesaba katar (düşük önyargı, yüksek varyans). $\lambda=0.95$ bu ikisi arasında dengeli bir noktadır.

PPO ile Eğittimiz Model 2

PPO_Sweetpot_V4 Modeli :

Aynı PPO algoritmasının kasıtlı olarak kısıtlanmış bir görüş yarıçapıyla (4 birim) ve daha düşük ve dinamik bir öğrenme katsayısıyla (5×10^{-5}) ince ayar yapılmış versiyonudur.

Diğer tüm hiperparametreler PPO Hardcore ile aynı tutulmuştur: $\gamma=0.99$, $\lambda=0.95$, entropi=0.015, clip=0.2. Öğrenme katsayısının düşürülmesi politika güncellemelerini daha temkinli hale getirirken, görüş alanının daraltılması modeli yakın çevresini daha dikkatli analiz etmeye yönlendirmiştir. Bu tasarım tercihlerinin sonucu olarak daha önce hiç karşılaşmadığı harita düzenlerine de başarıyla uyum sağlamış ve 500 özdeş harita üzerinde gerçekleştirilen karşılaştırmalı testte %91.9 ortalama başarı oranıyla dört model arasında genel şampiyon olmuştur

PPO Modelleri Arasındaki Fark

Parametre	PPO SWEETSPOT V4	PPO HARDCORE V2
Öğrenme Oranı	3×10^{-4} 'ten 5×10^{-5} 'e kademeli azalan	3×10^{-4} (6 kat yüksek)
Görüş Yarıçapı	4 birim (dar)	5 birim (geniş)
Toplam Eğitim Adımı	1.8 M	1.2 M

A2C ile Eğittimiz Model

A2C (Advantage Actor-Critic), tek bir sinir ağı üzerinde iki farklı görevi eş zamanlı yürüten bir aktör-eleştirmen mimarisine dayanmaktadır. Ağın aktör başlığı, ajanın hangi aksiyonu seçeceğini belirleyen politikayı üretirken; eleştirmen başlığı, mevcut durumun ne kadar değerli olduğunu tahmin etmektedir.

Eleştirmenin ürettiği değer tahmini, aktörün her adımda kendini ne yönde güncelleyeceğini belirleyen avantaj hesaplamasında kullanılmaktadır. Bu iki bileşenin birbirini karşılıklı olarak beslediği yapı, öğrenmenin hem hızlı hem de kararlı ilerlemesini mümkün kılmaktadır.

A2C ile Eğittimiz Model - devam..

Modelin en belirgin özelliği, eğitim sürecinde 8 bağımsız ortamın eş zamanlı olarak çalıştırılmasıdır. Her ortam farklı bir harita ve farklı bir rastgele tohum ile başlatılmakta; bu sayede tek bir ortamdan elde edilemeyecek çeşitlilikte deneyim aynı anda toplanmaktadır. Toplanan deneyimler tek bir senkron güncellemeye tabi tutulmakta, böylece asenkron yöntemlerde ortaya çıkabilen bayat gradyan sorununun önüne geçilmektedir.

Öğrenme katsayısı 3×10^{-4} , indirim faktörü $\gamma=0.99$ ve GAE lambda değeri $\lambda=0.95$ olarak belirlenmiştir.

Keşif davranışı, DQN'deki gibi epsilon ile değil; entropi katsayısı aracılığıyla yönetilmiştir.

Bu katsayı eğitimin başında 0.03 olarak ayarlanmış, politikanın erken kararlaşmasını önlemek amacıyla zamanla 0.01'e düşürülmüştür.

Ödül şemasında hedefe ulaşma +100, engele çarpma -50, harita dışına çıkma -10 olarak tanımlanmış; hedefe yaklaşma +2.0, uzaklaşma -0.5 ve her adıma -0.1 temel ceza uygulanmıştır.

SAC ile Eğittiğimiz Model

SAC (Soft Actor-Critic), örneklem verimliliğini maksimize eden off-policy (politika dışı) bir yapıyla entropi düzenlemesini bir araya getiren gelişmiş bir aktör-eleştirmen modelidir. Ajanın yalnızca beklenen ödülü değil, aynı zamanda politikasının rastgeleliğini (entropisini) de eş zamanlı olarak maksimize etmesini sağlayan bu yapı, yerel minimumlara saplanmayı engeller ve daha yaratıcı/esnek kaçış manevraları bulunmasını teşvik eder. Stable-Baselines3 kütüphanesi üzerinden eğitilmiş olan bu model, PPO ve A2C'den farklı olarak, ağdan çıkan eylemleri başlangıçta $[x, y]$ formatında sürekli (continuous) uzayda üretip, ardından çevre sarmalayıcısında 5 ayrık (discrete) aksiyona dönüştürerek çalışmaktadır.

Öğrenme katsayısı 3×10^{-4} , indirim faktörü $\gamma=0.99$ ve hedef ağların aşırı iyimser tahminlerini (overestimation) törpülemek amacıyla yumuşak güncelleme katsayısı $\tau=0.005$ olarak belirlenmiştir.

Keşif davranışı; DQN, PPO veya A2C'deki gibi dışarıdan manuel ayarlanan kademeli bir parametre yerine, ideal keşif seviyesini eğitim boyunca optimumda tutmak için algoritmanın kendi kendine ayarladığı "dinamik (auto) entropi katsayısı" ile yönetilmiştir.

Deneyimler 100.000 kapasiteli çok geniş bir bellek tamponunda (replay buffer) saklanmış ve eğitim süreci boyunca geçmiş deneyimlerden 256 boyutlu rastgele batch'ler (yığınlar) çekilerek ağ ağırlıkları off-policy olarak sürekli güncellenmiştir.

SAC ile Eğittiğimiz Model

Modelin eğitim sürecindeki en güçlü yönlerinden biri, 16 bağımsız ortamın (SubprocVecEnv) eş zamanlı olarak çalıştırılmasıdır. 7 birimlik geniş bir görüş yarıçapıyla çevresini analiz eden SAC modeli, PPO modelleriyle aynı gelişmiş ödül şemasını temel almıştır:

Sabit adım cezası -0.05 , yeni hücre keşfine $+0.3$, dinamik engele tehlikeli yakınlıkta -0.2 ve çarpışmada -50 uygulanmıştır.

Çıkarım aşamasında (inference) kararlarını tamamen kendi "saf sinir ağı" ile veren bu model, hiçbir manuel kural tabanlı kalkın kullanmadan zorlu senaryolarda hayatta kalmayı öğrenmiştir.

GENEL KARŞILAŞTIRILMA

PARAMETRE	DQN (V4)	A2C (V3)	PPO (SWEETSPOT)	PPO (HARDCORE)	SAC (LOCAL)
Algoritma Tipi	Value-Based	Actor-Critic	Policy-Based	Policy-Based	Off-Policy Entropi
Durum Temsili	16 Boyut	94 Boyut	102 Boyut	102 Boyut	102 Boyut
Görüş Yarıçapı	Yakın Çevre	7 Birim	4 Birim	5 Birim	7 Birim
Aksiyon Uzayı	4 (Ayrık)	4 (Ayrık)	5 (+Stay)	5 (+Stay)	5 (Ayrık/Box)
Toplam Adım	303.500	~19.95M (Total)	1.800.000	1.255.424	500.000
Öğrenme Oranı	1e-3	3e-4	5e-5 (Kademeli)	3e-4	3e-4
Benchmark Başarı	%82.6	%77.1	%91.9	Yüksek	Test Aşamasında

* PPO Sweetspot, kademeli azalan öğrenme oranı ve optimize edilmiş adım sayısı ile en yüksek genelleme başarısını göstermiştir.

Künyeden Finale Değişenler Ve Sebepleri

Projenin başında sistemi model, backend ve frontend olarak 3 ayrı repoya bölmüştük. Fakat her seferinde her reponun güncel halini çekip ayrı ayrı ayağa kaldırmak bizi epey yavaşlatmaya başladı. Biz de süreci hızlandırmak için tüm projeyi tek bir repo altında topladık ve bu sayede gerçekten ciddi bir hız kazandık.

Benzer bir değişikliği sunucu tarafında da yaptık. İlk başta backendi uzak sunucuya deploy edip haritaları da oradaki veri tabanında tutacaktık. Ancak projenin en kritik noktası olan Websocket (WS) gecikmelerinden çekindiğimiz için bu fikirden vazgeçtik. Harita verilerini lokalde hiç biriktirmeden, backendi doğrudan lokalde çalıştırdık. Böylece frontend, backend ve model arasındaki iletişimi gecikmesiz ve sorunsuz bir şekilde çözmüş olduk.

Künyeden Finale Değişenler Ve Sebepleri

Model kısmında ise yola sadece tek bir model ve DQN algoritmasıyla çıkmıştık. Sonradan projenin yönünü biraz değiştirerek, eksik veya doğru veri setleriyle eğitilmiş farklı modellerin performansını kıyaslamak istedik. Bu denemeleri yapabilmek için de projedeki algoritma ve model sayısını artırarak daha kapsamlı bir yapıya geçtik.

Eski repository'ler :

Model : <https://github.com/mertaydin10/otonomModel>

Frontend : <https://github.com/mertaydin10/OtonomSurus>

Backend : <https://github.com/civelekemirhan/PointSense>

Yeni repository (Fullstack) : <https://github.com/civelekemirhan/AutoSimulation>

TEŞEKKÜRLER